

Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники

Курсовая работа на тему
«Создание системы для контроля кондиционеров»
по дисциплине «Информационные системы»

Выполнила:
Денисова Алёна Александровна

Группа: Р3311

Проверил:
Тюрин Иван Николаевич

Санкт-Петербург, 2025

Содержание

1. Задание	3
1.1. Предметная область	3
1.2. Назначение и задачи системы	4
1.3. Функциональные и нефункциональные требования	4
1.4. Модели основных прецедентов	8
1.5. Архитектура будущей системы	10
2. Этап 2	10
2.1. Сформировать ER-модель базы данных	10
2.2. Реализовать даталогическую модель в реляционной СУБД PostgreSQL	11
2.3. Индексы PostgreSQL	12
2.3.1. Индекс на таблицу user по полю yandex_id	12
2.3.2. Индекс на таблицу schedule по полю air_conditioner_id . .	12
2.3.3. Составной индекс на таблицу schedule по полям (day_of_week, is_active)	12
2.4. Описание PL/pgSQL функций и процедур	13
2.4.1. Процедура register_user_with_role	13
2.4.2. Процедура set_group_ac_mode	13

1. Задание

1.1. Предметная область

Согласована предметная область: *система контроля кондиционеров*.

Система контроля кондиционеров предназначена для централизованного управления кондиционерами, установленных в различных зданиях и помещениях.

Основная цель: обеспечить удобный мониторинг и настройку работы климатического оборудования через веб-приложение.

Каждый кондиционер закрепляется за конкретным зданием и комнатой в этом здании.

Для кондиционеров предусмотрены следующие возможности управления:

- *Ручное управление* – включение/выключение, выбор режима работы, задание температуры.
- *Расписание работы* – пользователь может настроить временные интервалы, в которые кондиционер будет автоматически включаться или выключаться, а также задавать разные режимы работы в зависимости от времени суток или дня недели.
- *Мониторинг состояния* – отображение текущего статуса (включен/выключен, активный режим, установленная температура).

Система также интегрируется с температурными датчиками, установленными в помещениях. Эти датчики позволяют получать информацию о текущей температуре воздуха.

Для удобства пользователей предусмотрены:

- просмотр списка зданий, комнат и установленных в них кондиционеров;
- отображение информации о каждом кондиционере и датчике;
- возможность управлять отдельными кондиционерами или сразу группами (например, всеми устройствами в комнате или здании);
- централизованное редактирование и хранение расписаний.

Система поддерживает разграничение доступа через систему ролей:

- *Системный администратор* имеет полный доступ ко всем функциям системы, включая управление пользователями и их правами, просмотр системных метрик.
- *Администратор здания* имеет доступ только к зданиям, закрепленным за ним, и может управлять кондиционерами в этих зданиях.
- *Обычный пользователь* имеет ограниченные права просмотра информации о состоянии кондиционеров в доступных ему помещениях.

1.2. Назначение и задачи системы

Информационная система для контроля кондиционеров необходима для централизованного управления кондиционерами в зданиях и помещениях. Она упрощает процесс эксплуатации, позволяет эффективно организовать работу кондиционеров и обеспечивает экономию ресурсов за счет автоматизации.

Задачи, которые решает система:

- *Централизованное управление*: возможность управлять всеми кондиционерами через единое веб-приложение, без необходимости физического доступа к каждому устройству.
- *Автоматизация работы оборудования*: настройка расписаний включения и выключения кондиционеров.
- *Мониторинг состояния и параметров*: отображение текущего состояния кондиционеров; получение информации с температурных датчиков в реальном времени.
- *Оптимизация энергопотребления*: предотвращение работы кондиционеров вне необходимости.
- *Управление иерархией объектов*: структурированное хранение информации о зданиях, комнатах и установленных кондиционерах; групповые операции.

1.3. Функциональные и нефункциональные требования

Функциональные требования

- **FR01.** Система должна предоставлять возможность подачи заявки на получение аккаунта.
- **FR02.** Система должна предоставлять администраторам возможность одобрения аккаунта нового пользователя.

- **FR03.** Система должна ограничивать доступ к сервису не одобренным пользователям.
- **FR04.** Система должна предоставлять администраторам возможность добавлять здание и комнату.
- **FR05.** Система должна предоставлять администраторам возможность добавить кондиционер в комнату.
- **FR06.** Система должна предоставлять администраторам возможность включить или выключить кондиционер.
- **FR07.** Система должна предоставлять администраторам возможность выбрать режим работы кондиционера из существующих: охлаждение, обогрев, осушение, вентиляция, автоматический.
- **FR08.** Система должна предоставлять администраторам возможность создать расписание работы кондиционера. Расписание должно содержать следующие поля: название расписания, день недели или конкретную дату, время начала и окончания действия, режим работы кондиционера (охлаждение, обогрев, осушение, вентиляция, автоматический), целевую температуру, периодичность выполнения (однократно, ежедневно, еженедельно, ежемесячно), статус активности расписания.
- **FR09.** Система должна предоставлять администраторам возможность просматривать список зданий, комнат и кондиционеров.
- **FR10.** Система должна предоставлять администраторам возможность просматривать текущее состояние кондиционера, причем кондиционер может находиться в одном из четырех состояний: включен, выключен, сломан, на ремонте.
- **FR11.** Система должна предоставлять администраторам возможность редактировать и удалять расписания кондиционера.
- **FR12.** Система должна предоставлять системным администраторам возможность просмотра, визуализации и экспорта метрик.
- **FR13.** Система должна поддерживать разделение пользователей на роли: системный администратор, администратор здания и обычный пользователь.
- **FR14.** Система должна предоставлять системным администраторам возможность назначать роли пользователям и управлять их правами доступа.
- **FR15.** Система должна ограничивать доступ администраторов зданий только к закрепленным за ними зданиям и их комнатам.
- **FR16.** Система должна предоставлять возможность создавать группы кондиционеров для упрощения группового управления.

- **FR17.** Система должна предоставлять администраторам возможность выполнять операции с группами кондиционеров (включение/выключение всех устройств в группе одновременно).
- **FR18.** Система должна поддерживать интеграцию с тестовым стендом для эмуляции работы кондиционеров и температурных датчиков в процессе разработки и тестирования.

Нефункциональные требования

Usability (удобство использования)

- **U01.** Система должна обеспечивать единообразие дизайна и расположения элементов на всех страницах сайта.
- **U02.** Система должна предоставлять пользователям возможность быстро находить нужное здание, комнату или кондиционер через поиск или фильтры. Фильтрация может осуществляться по местоположению, состоянию и режиму. Пользователь может выбирать значение критерия фильтра из выпадающего списка.
- **U03.** Система должна предоставлять интерфейс отмены или подтверждения ключевых операций редактирования/удаления.

Reliability (надёжность)

- **R01.** Система должна сохранять все данные о кондиционерах и расписаниях при сбоях.
- **R03.** Система должна реализовывать возможность отмены или подтверждения ключевых операций редактирования/удаления.
- **R04.** Система должна предоставлять вход в систему с использованием Яндекс ID.
- **R05.** Система должна иметь возможность к развёртыванию на нескольких узлах сети.

Performance (производительность)

- **P01.** Система должна обеспечивать обработку 90% пользовательских запросов не более чем за 2 секунды.
- **P02.** Система должна предоставлять возможность обработки не менее 5000 запросов в сутки.
- **P03.** Система должна обеспечивать время загрузки главной страницы < 5 секунд для 95% пользователей.
- **P04.** Система должна предоставлять возможность обновления данных с датчиков температуры.

- **P05.** По завершении разработки должно быть проведено нагрузочное тестирование для верификации различных пользовательских сценариев и соответствия заявленным требованиям производительности.

Design Constraints (ограничения разработки)

- **D01.** Для разработки бэкенда использовать Java 24, Spring Web MVC, Spring Data JPA, Spring Security, Spring Session, Spring Boot, Lombok, Spring Boot Actuator, систему сборки Gradle.
- **D02.** Для авторизации использовать протокол OAuth 2.0 через API Яндекс ID.
- **D03.** В качестве СУБД использовать PostgreSQL. Для миграций баз данных использовать библиотеку Liquibase.
- **D04.** Для разработки фронтенда использовать TypeScript, библиотеку React, Tailwind CSS.
- **D05.** Для связи бэкенда и фронтенда использовать протокол HTTPS, RESTful API.
- **D06.** Для сбора метрик использовать систему Prometheus, Grafana для их визуализации.
- **D07.** В ходе разработки использовать версии библиотек, актуальные на сентябрь 2025 года.
- **D08.** Для контроля версий зависимостей фронтенда использовать npm.
- **D09.** Для описания API использовать спецификацию OpenAPI. На основе спецификации должны автоматически генерироваться каркас серверной части и клиентская часть для взаимодействия с сервером.
- **D10.** Для развёртывания системы использовать Docker и Docker compose.
- **D11.** Система должна автоматически разворачиваться при коммите в ветки main и dev репозитория.
- **D12.** Для консистентности стиля исходного кода на Backend-стороне использовать checkstyle
- **D13.** Для консистентности стиля исходного кода на Frontend-стороне использовать eslint
- **D14.** В репозитории должна быть настроена система непрерывной интеграции (CI), включающая для frontend и backend: сборку проекта, запуск форматтеров и линтеров в режиме проверки, запуск тестов.
- **D15.** Должен быть разработан тестовый стенд - отдельное Spring Boot приложение, эмулирующее работу кондиционеров и температурных датчиков. Стенд должен предоставлять REST API для управления виртуальными устройствами и имитировать их реальное поведение (изменение температуры, отклик на команды управления). Хранение

состояния устройств допускается в оперативной памяти без использования базы данных.

1.4. Модели основных прецедентов

Прецедент: Регистрация пользователя в системе
ID: 1
Краткое описание: Пользователь создаёт новый аккаунт для работы в системе.
Главные акторы: Пользователь
Предусловия: Пользователь не зарегистрирован в системе.
Основной поток: 1. Пользователь отправляет заявку на регистрацию, авторизуясь через Яндекс ID. 2. Система сохраняет пользователя в базе данных, выставляя статус PENDING на запрос регистрации. 3. Администратор одобряет регистрацию пользователя. Статус в базе данных меняется на APPROVED. 4. Пользователь получает доступ к системе.
Постусловия: Новый пользователь получил доступ к системе.

Прецедент: Управление кондиционером
ID: 2
Краткое описание: Администратор вручную управляет состоянием кондиционера.
Главные акторы: Администратор
Предусловия: Администратор вошёл в систему и имеет доступ к комнате с кондиционером.
Основной поток: 1. Администратор выбирает необходимый кондиционер.

2. Администратор задаёт действие: включить/выключить, изменить режим или температуру. 3. Система отправляет команду на устройство. 4. Система обновляет интерфейс и отображает новое состояние.
Постусловия: Параметры кондиционера изменены в соответствии с действиями администратора.

Прецедент: Создание расписания работы кондиционера
ID: 3
Краткое описание: Администратор автоматизирует работу кондиционера через расписание.
Главные акторы: Администратор
Предусловия: Администратор аутентифицирован, кондиционер добавлен в систему.
Основной поток: 1. Администратор выбирает необходимый кондиционер из списка. 2. Администратор нажимает кнопку “Добавить расписание”. 3. Администратор выбирает параметры расписания. 4. Система сохраняет расписание для кондиционера.
Постусловия: В системе создано расписание для выбранного кондиционера.

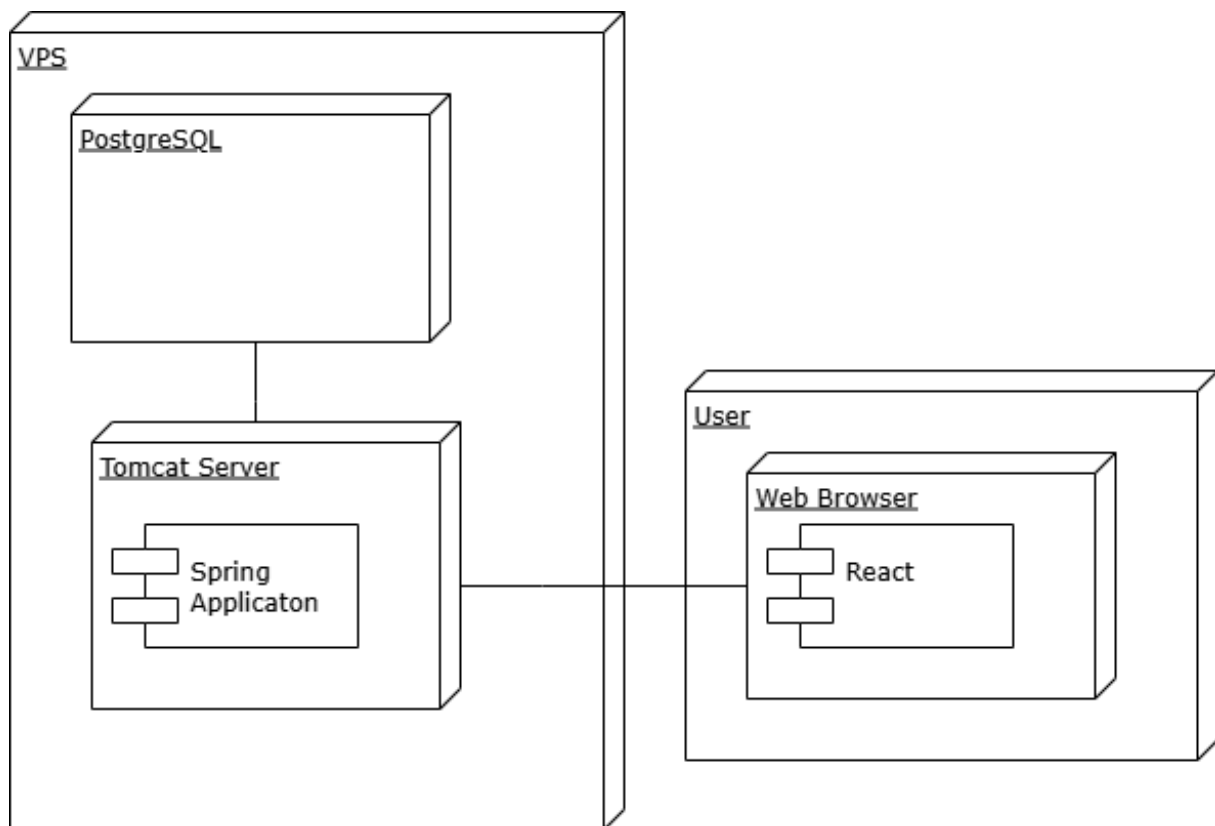
Прецедент: Получение данных с датчиков температуры
ID: 4
Краткое описание: Система получает и отображает актуальные показания датчиков.
Главные акторы: Система, Датчик температуры
Предусловия: Датчики подключены и передают данные.
Основной поток:

1. Датчик фиксирует текущую температуру.
2. Система запрашивает данные с датчика.
3. Система принимает данные и обновляет значения в базе.
4. Система отображает данные пользователю в интерфейсе.

Постусловия: Пользователь видит актуальную температуру в выбранной комнате.

1.5. Архитектура будущей системы

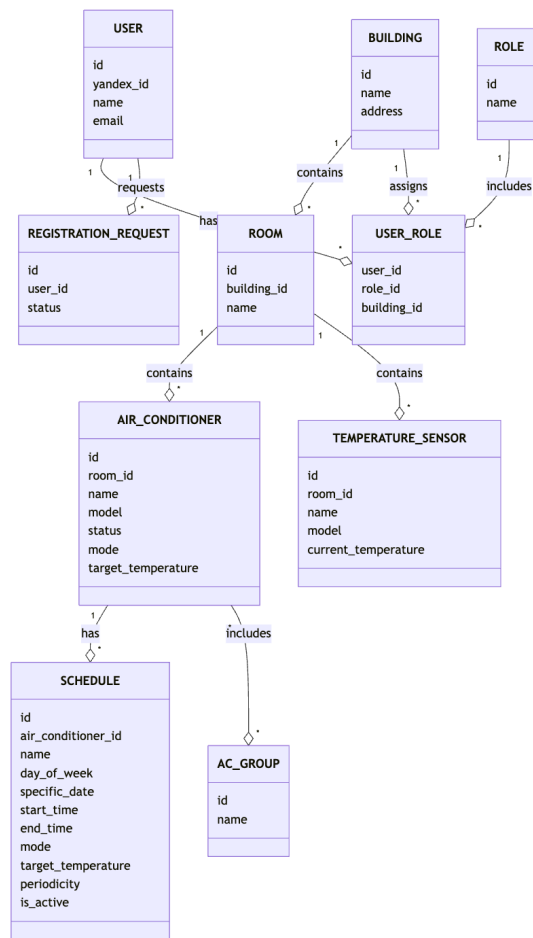
Диаграмма развертывания:



2. Этап 2

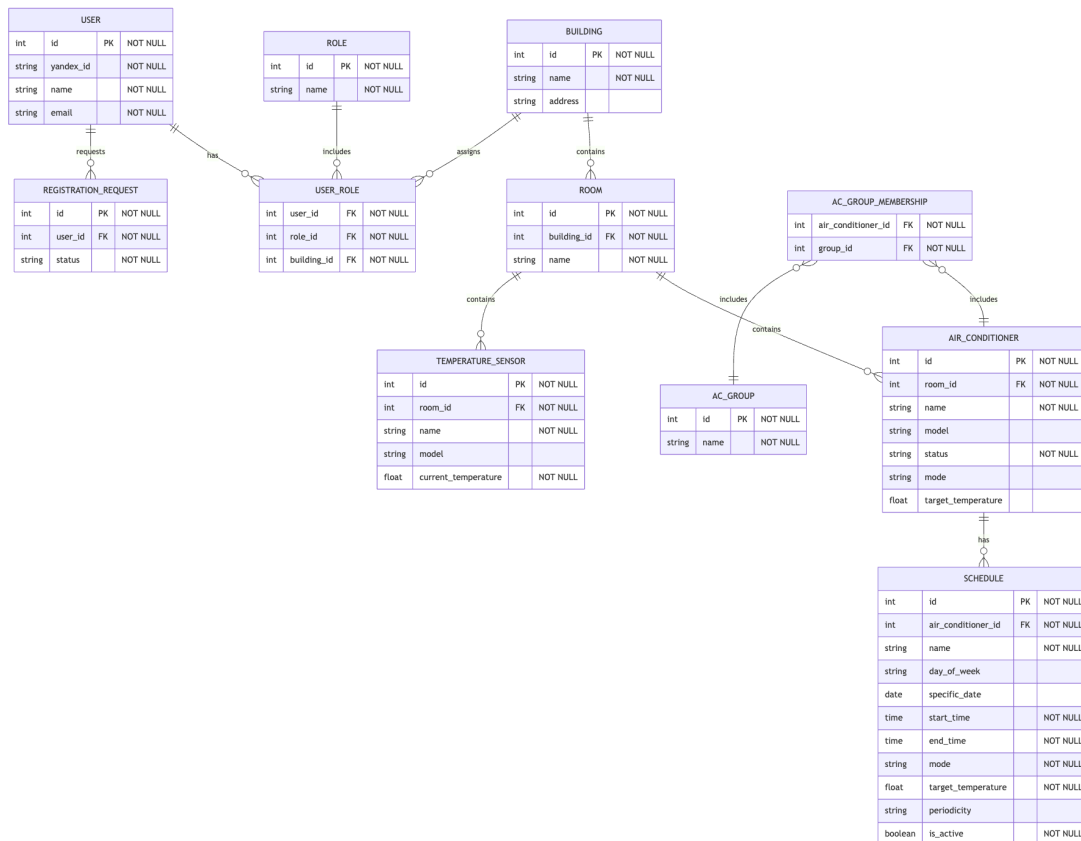
2.1. Сформировать ER-модель базы данных

В рамках второго этапа была сформирована ER-модель предметной области, представляющая собой концептуальную схему системы управления кондиционерами. Модель включает все основные сущности предметной области и связи между ними.



2.2. Реализовать даталогическую модель в реляционной СУБД PostgreSQL

На основе ER-модели была реализована даталогическая модель в реляционной СУБД PostgreSQL.



2.3. Индексы PostgreSQL

Для оптимизации производительности запросов к базе данных были добавлены следующие индексы:

2.3.1. Индекс на таблицу user по полю yandex_id

```
CREATE INDEX IF NOT EXISTS idx_user_yandex_id ON
"user" (yandex_id);
```

Этот индекс ускоряет аутентификацию пользователей через Яндекс ID, так как поиск по этому полю происходит при каждом входе в систему.

2.3.2. Индекс на таблицу schedule по полю air_conditioner_id

```
CREATE INDEX IF NOT EXISTS idx_schedule_air_conditioner_id ON
schedule (air_conditioner_id);
```

Индекс ускоряет получение расписаний для конкретного кондиционера, что необходимо при управлении расписаниями устройств.

2.3.3. Составной индекс на таблицу schedule по полям (day_of_week, is_active)

```
CREATE INDEX IF NOT EXISTS idx_schedule_day_active ON schedule
(day_of_week, is_active);
```

Составной индекс оптимизирует выборку активных расписаний по дням недели, что используется планировщиком для определения устройств, которые должны быть включены в определенный день.

2.4. Описание PL/pgSQL функций и процедур

Для оптимизации выполнения критически важных запросов и обеспечения целостности данных были разработаны специальные PL/pgSQL функции и процедуры.

2.4.1. Процедура register_user_with_role

Процедура register_user_with_role предназначена для регистрации нового пользователя в системе с одновременным назначением ему роли в конкретном здании. Например, при первоначальной настройке системы для нового здания или при форс-добавлении юзера (без отправления заявки с его стороны).

Параметры:

- yandex_id (VARCHAR) - уникальный идентификатор пользователя в Яндексе
- name (VARCHAR) - имя пользователя
- email (VARCHAR) - электронная почта пользователя
- role_id (INTEGER) - идентификатор роли (из таблицы role)
- building_id (INTEGER) - идентификатор здания (из таблицы building), в котором назначается роль

Что делает процедура:

1. Создает новую запись в таблице user с предоставленными данными
2. Назначает пользователю указанную роль в конкретном здании (запись в таблице user_role)
3. Создает заявку на регистрацию со статусом “approved” в таблице registration_request

2.4.2. Процедура set_group_ac_mode

Процедура set_group_ac_mode предназначена для массового изменения режима работы всех кондиционеров в группе.

Параметры:

- group_id (INTEGER) - идентификатор группы кондиционеров
- new_mode (VARCHAR) - новый режим работы кондиционеров
- new_target_temp (FLOAT) - новая целевая температура

Что делает процедура:

1. Обновляет режим работы и целевую температуру для всех кондиционеров в указанной группе
2. Автоматически изменяет статус кондиционеров в зависимости от установленного режима (например, переводит в неактивное состояние при установке режима “off”)